COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

09

Start the next bundle

Setup, readiness and study route

PACKAGE 09 / VOLUME 08 / SEPTEMBER 28, 2026
Modules 36–40 · Gate H · Expanded teaching edition 1.0
Study, implement, test and review. Models are not physical measurements.

Contents

Package 09 expands Volume 08 3
What is included 3
Prerequisites and workload 3
Set up the software-only path 4
First focused session 5
Completion and feedback 5
Reproduce the reference case 5
Troubleshooting 6

Package 09 expands Volume 08

This is **Gaming Platforms and Hardware/Software Co-design**, the expanded eighth volume in the Computer Science and Computer Engineering for Games program. It preserves Modules 36–40, lessons 36.1–40.4, their learning outcomes, original assignment briefs, assigned readings and the original three-question lesson quizzes. Each lesson now adds a deeper explanation, worked examples, a runnable computational slice, a full practical route and two additional application questions.

Package and volume numbers differ. Package 01 was the initial program; Package 09 is the expansion of Volume 08. The next volume is Graduate Game-Engine and Graphics Engineering. This package does not replace the earlier mathematics, C++, systems, graphics or digital-design prerequisites.

What is included

The teaching set contains 20 expanded lessons, 20 runnable demonstrations, 20 student starter/reference pairs, 100 lesson quiz questions, 25 module-review questions, 40 written calculation/design problems, ten entry diagnostic questions, five module projects and Gate H. The 100 lesson questions consist of 60 original questions and 40 additions. Diagnostic, review and written-problem counts are separate.

The Student Handbook teaches the concepts. The Lab Workbook preserves each original practical assignment and separates it from the narrower automatically checked slice. The Answer Key includes explanations for all lesson questions, reviews, written problems and diagnostic questions. The Capstone Specification defines the learner's final portfolio. The Platform Reference provides units, formulas, contract checklists and a glossary. REFERENCE_CASE_STUDY reports the supplied executable teaching example, not a completed learner capstone.

Open COURSE.html after extracting the entire ZIP for an offline browser edition with navigation, text search and answers collapsed by default. External reading links still require internet access. Markdown and JSON sources are supplied for annotation and use by local tools or AI agents. Keep the original manifest and original quiz markers when adapting this course.

Prerequisites and workload

The ordinary route requires the earlier systems, graphics and digital-design work, including Gates F and G before Gate H. Follow each module's explicit prerequisites. You should be able to write and test a small Python/C++ program, reason about binary representation and arrays, distinguish clocks and deadlines, explain basic CPU/GPU memory behavior, and work with rigid camera transforms.

The original planning estimate is **350–560 hours** including external reading, exercises, implementation, revision, projects and review. It is not the time required to read these PDFs and is not academic credit. A suggested allocation is below; module projects feed Gate H rather than becoming five unrelated capstones.

Module	Suggested planning range	Main evidence
36	70–110 hours	Peripheral, scheduler, schematic/ layout and review

Module	Suggested planning range	Main evidence
37	60–100 hours	GPU architecture and memory workbook
38	70–110 hours	Reproducible performance investigation
39	50–80 hours	Constrained platform and latency report
40	100–160 hours	Integrated co-design portfolio and review

These allocations are planning choices within the original range, not validated completion times. At ten hours per week the total range spans approximately 35–56 weeks; prior competence can change the route. Work at the pace that produces independent implementation and explanation.

Set up the software-only path

Extract the ZIP to a writable folder and keep its layout intact. The Python code uses the standard library, with Python 3.11+ as its chosen syntax baseline. Automated execution for this edition used Python 3.13.5 on Linux. Other operating systems and Python versions were not separately executed here. On macOS/Linux the interpreter may be called `python3`; on Windows the launcher may be `py`. Replace `python` consistently in the examples with the command available on your machine.

The core model exercises require no GPU, FPGA, game-engine installation, root privileges, paid account or proprietary SDK. The original board-design assignment still requires learning KiCad and producing reviewable schematic/layout files; the small JSON linter does not replace that work. Physical fabrication and hardware bring-up remain optional and require qualified review. A locally installed C++17 compiler is needed only for the optional native reference benchmark. The package never installs packages, changes firmware or loads device drivers.

From a terminal whose current directory is the extracted Game_Engineering_Package_09 folder:

```
python --version
python -m labs.run list
python -m labs.run demo 36.1
python -m labs.run check 36.1
```

The last command checks the supplied reference, not your learning. Edit `labs/starters/lesson_36_1.py` and then run:

```
python -m labs.run check 36.1 --student
```

The untouched starter intentionally raises NotImplementedError and therefore fails. Keep the documented `solve(case)` interface and returned field names. Add tests beyond the published cases; passing a small known suite is necessary evidence, not proof of complete correctness. Open the reference only after an independent attempt.

First focused session

Spend the first 15 minutes reading this guide and answering the entry diagnostic in the workbook. Spend the next 20 minutes writing the 36.1 electrical/transport/application contract without running code. Predict the five-byte reply, byte order, signed value, checksum and timeout behavior on paper. Then run the demo for about ten minutes and compare its output with your prediction.

Use the next 25–35 minutes to implement the starter and test malformed input, equality at the deadline and one late response. Finish by answering the five lesson questions without opening the key. Log one misconception, the evidence that corrected it and the remaining full-assignment work. Do not treat a 90-minute session as completion of all peripheral engineering.

Completion and feedback

A lesson is complete when you have completed the assigned reading, explained the worked example, attempted all five quiz questions, completed the original practical assignment with its pass criteria, and saved the required evidence. The automatically checked slice is only one part. Record external source/section/exercise identifiers in your study log. The supplied questions and problems are a starting bank, not a replacement for all university-level practice in these subjects.

For self-checking, score each lesson question 0, 1 or 2: absent/incorrect; partly correct but missing a key condition; correct with the relevant boundary or assumption. A suggested readiness target is 8/10 after correction, with no unresolved safety or ownership misconception. This is a local study rule, not a validated examination standard. Reattempt missed questions later without memorizing the answer key.

Use PROGRESS.md to distinguish reading, quiz, computational slice, full practical work, project and review. A peer should reproduce each module project and challenge at least one major claim. The Gate H rubric has mandatory correctness, ownership and evidence conditions that cannot be offset by a high arithmetic score.

Reproduce the reference case

The core reference run needs Python only. Use a new output folder to preserve the supplied edition's example measurements:

```
python -m capstone.run_capstone --output my_results/model_only
python -m capstone.audit my_results/model_only/evidence_ledger.json
```

With an existing C++17 compiler:

```
python -m capstone.run_capstone --native --output my_results/native
python -m capstone.audit my_results/native/evidence_ledger.json
```

The native path compiles the included source and executes ordinary host code. It does not execute an FPGA or GPU accelerator. All model clocks, bandwidths and multiplier counts remain assumptions or structural estimates. The optional native timings are host function measurements, not input-to-photon or whole-game results. Small native cases can be dominated by timing overhead; inspect the method before interpreting nanoseconds.

Troubleshooting

If Python reports `No module named labs`, open the terminal in the extracted package root, not in the ZIP preview or inside `labs`. If a student test fails with NotImplementedError, the starter has not yet been implemented. If a return dictionary differs from the checker, compare the documented contract and expected keys rather than changing the expected answer to fit your code. For --native compiler errors, keep the log, check that a C++17 compiler is already installed, or omit --native and continue with the supported Python-only model route.

If the browser edition's search is unavailable, use the browser's Find function or the searchable PDFs. No course component requires enabling an external service. For KiCad, graphics APIs or device extensions, use official documentation matching the installed version; the reading directory includes versioned and moving documentation rather than claiming one universal current configuration.